Principal Component Analysis and Independent Component Analysis

From Variance Maximization to Independent Sources

OMSCS ML | Continuous Latent Variables Unit

Table of Contents

1. Motivation: Low-Dimensional Structure in High-Dimensional Data
2. PCA: Maximum Variance Formulation
   - 2.1 The Setup
   - 2.2 Deriving the First Principal Component
   - 2.3 Additional Components by Induction
3. PCA: Minimum Reconstruction Error Formulation
   - 3.1 The Setup
   - 3.2 Optimizing the Projection Coordinates
   - 3.3 Optimizing the Basis Directions
   - 3.4 Why Both Formulations Agree
4. Applications of PCA
   - 4.1 Data Compression and Visualization
   - 4.2 Whitening (Sphereing)
5. Probabilistic PCA
   - 5.1 Motivation and the PPCA-to-VAE Arc
   - 5.2 The Generative Model
   - 5.3 The Marginal Distribution $p(\mathbf{x})$
   - 5.4 Maximum Likelihood Solution
   - 5.5 The Posterior $p(\mathbf{z} \mid \mathbf{x})$ and the Latent Projection
   - 5.6 EM for PPCA (Pointer)
   - 5.7 Factor Analysis: One Structural Difference
6. From PCA to ICA: Why Non-Gaussianity Matters
   - 6.1 PCA's Fundamental Limitation: Rotational Ambiguity
   - 6.2 The Cocktail Party Problem
   - 6.3 The ICA Model
   - 6.4 Ambiguities Inherent in ICA
   - 6.5 Why Gaussian Latents Are Forbidden
   - 6.6 Independence vs. Uncorrelatedness
7. ICA Estimation: Finding Independent Components
   - 7.1 The Core Idea: Non-Gaussianity as a Target
   - 7.2 Kurtosis
   - 7.3 Negentropy
   - 7.4 Mutual Information and the Infomax Principle
   - 7.5 Preprocessing: Centering and Whitening
   - 7.6 The FastICA Algorithm
8. Sources and Further Reading

1. Motivation: Low-Dimensional Structure in High-Dimensional Data

A recurring theme in machine learning is that high-dimensional data often lives on a much lower-dimensional subspace — or at least stays close to one. The data's *observed* dimensionality (D) is the number of features we measure; its *intrinsic* dimensionality is the number of degrees of freedom actually

driving variation.

A concrete example. Consider a dataset of portrait photographs, each stored as a 256×256 grayscale image — $D = 65{,}536$ dimensions. Yet the space of natural-looking faces is controlled by a much smaller number of factors: lighting direction, head pose (left-right tilt, up-down), facial expression, and a set of identity-specific features. Empirically, somewhere between 50 and 200 directions account for almost all meaningful variation in a large face dataset; the remaining tens of thousands of pixel dimensions are largely redundant once those factors are fixed. The high dimensionality is an artifact of the pixel representation, not of the underlying structure.

The same pattern appears in motion capture data. Recording an actor walking attaches 50 markers to the body and tracks their 3D positions — $D = 150$ dimensions. But walking is a periodic, mechanically constrained motion: once you know the gait phase and a few parameters like stride length and speed, the positions of all 150 coordinates are largely determined. The intrinsic dimensionality is perhaps a handful; the rest is noise and redundancy.

Why does this matter? Two reasons. First, *computational efficiency*: algorithms that scale with D are expensive when D is large, but if the true structure is M-dimensional with $M \ll D$, we can work in the lower-dimensional space. Second, *generalization*: a model that exploits the manifold structure concentrates probability mass where data actually appears, rather than spreading it diffusely across all of $\mathbb{R}^D$.

The modeling strategy. Assume each observed data point $\mathbf{x} \in \mathbb{R}^D$ is generated from a lower-dimensional latent variable $\mathbf{z} \in \mathbb{R}^M$ with $M < D$, via some (possibly noisy) mapping. Finding this mapping — and the values of $\mathbf{z}$ for each observation — is the problem these notes address.

We start with the simplest possible version: a *linear* mapping and *Gaussian* latent distribution. This is PCA and its probabilistic generalization. We then ask what happens when we relax the Gaussian assumption, which leads to ICA.

2. PCA: Maximum Variance Formulation

2.1 The Setup

Notation. We have N data points $\{\mathbf{x}_n\}_{n=1}^N$ with $\mathbf{x}_n \in \mathbb{R}^D$. We want to find a direction $\mathbf{u}_1 \in \mathbb{R}^D$ to project onto — intuitively, the direction along which the data varies most.

Define the sample mean:

$$\bar{\mathbf{x}} = \frac{1}{N}\sum_{n=1}^{N} \mathbf{x}_n$$

and the sample covariance matrix:

$$\mathbf{S} = \frac{1}{N}\sum_{n=1}^{N} (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T \in \mathbb{R}^{D\times D}$$

Note that $\mathbf{S}$ is symmetric ($\mathbf{S}^T = \mathbf{S}$) and positive semidefinite. Both properties will matter shortly.

Why maximize variance? Picture dropping 2D data points perpendicularly onto a number line — each point lands at a single scalar value, its projection. If the line runs through the main spread of the data, the projected points land far apart and remain distinguishable. If the line runs perpendicular to the spread, all points collapse near the same value and the structure is lost. Maximizing the variance of the projected values is precisely the criterion that keeps the projected points as spread out and distinguishable as possible.

The normalization convention. We are searching for a *direction*, not a vector with a particular magnitude. We therefore constrain $\mathbf{u}_1$ to be a unit vector: $\mathbf{u}_1^T\mathbf{u}_1 = 1$. Without this, the optimization is unbounded — we could scale $\|\mathbf{u}_1\| \to \infty$ to inflate the projected variance arbitrarily.

The projection. Each data point $\mathbf{x}_n$ projects onto the scalar $\mathbf{u}_1^T\mathbf{x}_n$. The mean of the projected values is $\mathbf{u}_1^T\bar{\mathbf{x}}$, and the variance of the projected values is:

$$\frac{1}{N}\sum_{n=1}^{N}\left(\mathbf{u}_1^T\mathbf{x}_n - \mathbf{u}_1^T\bar{\mathbf{x}}\right)^2 = \frac{1}{N}\sum_{n=1}^{N}\left(\mathbf{u}_1^T(\mathbf{x}_n - \bar{\mathbf{x}})\right)^2 = \mathbf{u}_1^T\mathbf{S}\mathbf{u}_1$$

The last step rewrites each squared scalar as a quadratic form: for $\mathbf{v}_n = \mathbf{x}_n - \bar{\mathbf{x}}$, we have $(\mathbf{u}_1^T\mathbf{v}_n)^2 = \mathbf{u}_1^T(\mathbf{v}_n\mathbf{v}_n^T)\mathbf{u}_1$. Summing over n and dividing by N gives $\mathbf{u}_1^T\mathbf{S}\mathbf{u}_1$ by definition of $\mathbf{S}$.

So the problem is:

$$\max_{\mathbf{u}_1} \ \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 \qquad \text{subject to} \qquad \mathbf{u}_1^T \mathbf{u}_1 = 1$$

This is a Lagrange-multiplier problem. (See the Lagrange Multipliers notes for the general method; here we apply it directly.)

2.2 Deriving the First Principal Component

Forming the Lagrangian. Introduce a multiplier λ_1 for the constraint $\mathbf{u}_1^T \mathbf{u}_1 - 1 = 0$. The Lagrangian is:

$$\mathcal{L}(\mathbf{u}_1, \lambda_1) = \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 + \lambda_1 \left(1 - \mathbf{u}_1^T \mathbf{u}_1\right)$$

Taking the gradient. Differentiate with respect to $\mathbf{u}_1$ and set to zero. For the first term, we use the identity $\nabla_{\mathbf{u}}(\mathbf{u}^T \mathbf{A} \mathbf{u}) = 2\mathbf{A}\mathbf{u}$, which holds whenever $\mathbf{A}$ is symmetric — which $\mathbf{S}$ is. To see why, write $f(\mathbf{u}) = \mathbf{u}^T \mathbf{A} \mathbf{u} = \sum_i \sum_j u_i A_{ij} u_j$ and differentiate with respect to a single component u_k. By the product rule, the kth partial derivative picks up two contributions — once when $i = k$ and once when $j = k$:

$$\frac{\partial f}{\partial u_k} = \sum_j A_{kj} u_j + \sum_i u_i A_{ik} = (\mathbf{A}\mathbf{u})_k + (\mathbf{A}^T \mathbf{u})_k$$

When $\mathbf{A}$ is symmetric ($\mathbf{A} = \mathbf{A}^T$), both terms are identical, giving $\partial f / \partial u_k = 2(\mathbf{A}\mathbf{u})_k$. Stacking all k gives $\nabla_{\mathbf{u}} f = 2\mathbf{A}\mathbf{u}$. For the second term, $\nabla_{\mathbf{u}}(\mathbf{u}^T \mathbf{u}) = 2\mathbf{u}$. Setting the combined gradient to zero:

$$2\mathbf{S}\mathbf{u}_1 - 2\lambda_1 \mathbf{u}_1 = \mathbf{0}$$

$$\boxed{\mathbf{S}\mathbf{u}_1 = \lambda_1 \mathbf{u}_1}$$

This is the eigenvector equation for $\mathbf{S}$: at any stationary point of the constrained problem, $\mathbf{u}_1$ must be an eigenvector of the data covariance matrix with eigenvalue λ_1.

Identifying the maximum. Stationary points exist wherever $\mathbf{u}_1$ is any eigenvector of $\mathbf{S}$ — but which one maximizes the projected variance? Left-multiply the eigenvector equation by $\mathbf{u}_1^T$:

$$\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 = \lambda_1 \underbrace{\mathbf{u}_1^T \mathbf{u}_1}_{=1} = \lambda_1$$

The projected variance *equals the eigenvalue*. To see why this is meaningful: $\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1$ is a quadratic form — it applies $\mathbf{S}$ to $\mathbf{u}_1$ to get a new vector $\mathbf{S}\mathbf{u}_1$, then asks how much of that vector points in the direction of $\mathbf{u}_1$. When $\mathbf{u}_1$ is an eigenvector, $\mathbf{S}\mathbf{u}_1 = \lambda_1 \mathbf{u}_1$, so the answer is simply λ_1 — the eigenvalue is the factor by which $\mathbf{S}$ stretches $\mathbf{u}_1$ along its own direction, which is exactly the variance of the data in that direction. Choosing the direction of largest stretch therefore means choosing the direction of maximum data spread. Since $\mathbf{S}$ is positive semidefinite all eigenvalues are ≥ 0, and the maximum is achieved by choosing $\mathbf{u}_1$ to be the eigenvector corresponding to the **largest eigenvalue** λ_1 of $\mathbf{S}$. This eigenvector is called the **first principal component**.

So what? The algebra has revealed something geometrically meaningful: the direction of maximum data spread is precisely the top eigenvector of the covariance matrix. The eigenvectors of $\mathbf{S}$ are the natural coordinate axes of the data's variance structure, and the eigenvalues tell us how much variance is captured in each direction. The covariance matrix *encodes* all pairwise spread information, and its eigenvectors diagonalize it, producing a coordinate system where the variables are uncorrelated.

2.3 Additional Components by Induction

For an M-dimensional projection, we want M orthonormal directions $\mathbf{u}_1, \ldots, \mathbf{u}_M$ that jointly maximize total projected variance. The argument extends by induction (see Bishop Exercise 12.1 for the full proof):

- Assume the first m directions are the top-m eigenvectors of $\mathbf{S}$.
- To find the $(m+1)$th direction, maximize projected variance subject to normalization *and* orthogonality to all previous directions. Enforce these constraints via Lagrange multipliers.
- Using the orthonormality of the existing eigenvectors, the cross-constraint terms vanish and one again arrives at $\mathbf{S}\mathbf{u}_{m+1} = \lambda_{m+1} \mathbf{u}_{m+1}$.
- The maximum is achieved by the $(m+1)$th largest eigenvector.

The PCA result: The optimal M-dimensional linear projection that maximizes variance of the projected data is the projection onto the M eigenvectors $\mathbf{u}_1, \ldots, \mathbf{u}_M$ of $\mathbf{S}$ corresponding to the M largest eigenvalues $\lambda_1 \geq \cdots \geq \lambda_M$. The resulting **principal subspace** is unique when eigenvalues are distinct.

3. PCA: Minimum Reconstruction Error Formulation

There is an equivalent way to derive PCA that gives direct intuition for what the projection *costs* when we discard dimensions. Instead of asking "which direction preserves the most variance?", we ask "which direction loses the least information when we project?"

3.1 The Setup

Introduce a complete orthonormal basis $\{\mathbf{u}_i\}_{i=1}^D$ for $\mathbb{R}^D$, satisfying $\mathbf{u}_i^T\mathbf{u}_j = \delta_{ij}$ (the Kronecker delta: 1 if $i = j$, else 0).

Because the basis is complete, every data point can be represented *exactly* as a linear combination of the basis vectors:

$$\mathbf{x}_n = \sum_{i=1}^{D} \alpha_{ni}\,\mathbf{u}_i$$

for some coefficients α_{ni}. To find what those coefficients must be, take the inner product of both sides with a specific basis vector $\mathbf{u}_j$:

$$\mathbf{u}_j^T\mathbf{x}_n = \mathbf{u}_j^T \sum_{i=1}^{D} \alpha_{ni}\,\mathbf{u}_i = \sum_{i=1}^{D} \alpha_{ni}\,\underbrace{\mathbf{u}_j^T\mathbf{u}_i}_{=\delta_{ij}}$$

The orthonormality condition kills every term in the sum except $i = j$, leaving:

$$\mathbf{u}_j^T\mathbf{x}_n = \alpha_{nj}$$

So the coefficient of $\mathbf{u}_j$ in the expansion is simply the projection of $\mathbf{x}_n$ onto $\mathbf{u}_j$. Substituting back:

$$\mathbf{x}_n = \sum_{i=1}^{D} \left(\mathbf{x}_n^T\mathbf{u}_i\right)\mathbf{u}_i$$

We want to *approximate* $\mathbf{x}_n$ using only the first M basis vectors, plus fixed offsets for the remaining $D - M$:

$$\tilde{\mathbf{x}}_n = \underbrace{\sum_{i=1}^{M} z_{ni}\mathbf{u}_i}_{\text{data-point-specific}} + \underbrace{\sum_{i=M+1}^{D} b_i\mathbf{u}_i}_{\text{shared across all points}}$$

Here z_{ni} varies per data point (capturing what is individual about $\mathbf{x}_n$), and b_i is a constant shared across all data points (our single best guess for the discarded directions). We choose the $\{\mathbf{u}_i\}$, $\{z_{ni}\}$, and $\{b_i\}$ to minimize the average squared reconstruction error:

$$J = \frac{1}{N}\sum_{n=1}^{N} \|\mathbf{x}_n - \tilde{\mathbf{x}}_n\|^2$$

3.2 Optimizing the Projection Coordinates

Minimizing over z_{nj} for the kept directions. Write out J explicitly with the approximation substituted:

$$J = \frac{1}{N}\sum_{n=1}^{N} \left\| \mathbf{x}_n - \sum_{i=1}^{M} z_{ni}\mathbf{u}_i - \sum_{i=M+1}^{D} b_i\mathbf{u}_i \right\|^2$$

Differentiating with respect to z_{nj} (using the chain rule $\frac{\partial}{\partial z_{nj}}\|\mathbf{e}\|^2 = 2\mathbf{e}^T\frac{\partial \mathbf{e}}{\partial z_{nj}}$ where $\mathbf{e} = \mathbf{x}_n - \tilde{\mathbf{x}}_n$, and noting $\frac{\partial \tilde{\mathbf{x}}_n}{\partial z_{nj}} = \mathbf{u}_j$ since only the jth kept term depends on z_{nj}):

$$\frac{\partial J}{\partial z_{nj}} = \frac{2}{N}(\tilde{\mathbf{x}}_n - \mathbf{x}_n)^T\mathbf{u}_j = 0$$

Setting this to zero gives $\tilde{\mathbf{x}}_n^T\mathbf{u}_j = \mathbf{x}_n^T\mathbf{u}_j$. Now expand the left side using the definition of $\tilde{\mathbf{x}}_n$ and orthonormality:

$$\tilde{\mathbf{x}}_n^T\mathbf{u}_j = \left(\sum_{i=1}^{M} z_{ni}\mathbf{u}_i + \sum_{i=M+1}^{D} b_i\mathbf{u}_i\right)^T \mathbf{u}_j = \sum_{i=1}^{M} z_{ni}\underbrace{\mathbf{u}_i^T\mathbf{u}_j}_{=\delta_{ij}} + \sum_{i=M+1}^{D} b_i\underbrace{\mathbf{u}_i^T\mathbf{u}_j}_{=0} = z_{nj}$$

The second sum vanishes because $j \le M$ and the discarded basis vectors are orthogonal to $\mathbf{u}_j$. Therefore:

$$z_{nj} = \mathbf{x}_n^T \mathbf{u}_j$$

The optimal per-point coefficient is the projection of $\mathbf{x}_n$ onto $\mathbf{u}_j$.

Minimizing over b_j for the discarded directions. By the same chain rule, with $\frac{\partial \tilde{\mathbf{x}}_n}{\partial b_j} = \mathbf{u}_j$ (only the jth discarded term depends on b_j):

$$\frac{\partial J}{\partial b_j} = \frac{2}{N} \sum_{n=1}^{N} (\tilde{\mathbf{x}}_n - \mathbf{x}_n)^T \mathbf{u}_j = 0$$

By the same orthonormality argument, $\tilde{\mathbf{x}}_n^T \mathbf{u}_j = b_j$ for $j > M$. So:

$$\frac{1}{N} \sum_{n=1}^{N} (b_j - \mathbf{x}_n^T \mathbf{u}_j) = 0 \implies b_j = \frac{1}{N} \sum_{n=1}^{N} \mathbf{x}_n^T \mathbf{u}_j = \bar{\mathbf{x}}^T \mathbf{u}_j$$

The optimal constant for a discarded direction is the projection of the sample mean onto that direction — the best single guess for an entire dimension, in the least-squares sense.

The displacement vector. Now substitute both optimal solutions back and compute the error explicitly. Using the exact expansion $\mathbf{x}_n = \sum_{i=1}^{D} (\mathbf{x}_n^T \mathbf{u}_i) \mathbf{u}_i$:

$$\mathbf{x}_n - \tilde{\mathbf{x}}_n = \sum_{i=1}^{D} (\mathbf{x}_n^T \mathbf{u}_i) \mathbf{u}_i - \sum_{i=1}^{M} z_{ni} \mathbf{u}_i - \sum_{i=M+1}^{D} b_i \mathbf{u}_i$$

Split the first sum at M and group terms:

$$= \sum_{i=1}^{M} \underbrace{(\mathbf{x}_n^T \mathbf{u}_i - z_{ni})}_{=0} \mathbf{u}_i \; + \sum_{i=M+1}^{D} (\mathbf{x}_n^T \mathbf{u}_i - b_i) \mathbf{u}_i$$

The kept-direction terms vanish because $z_{ni} = \mathbf{x}_n^T \mathbf{u}_i$. Substituting $b_i = \bar{\mathbf{x}}^T \mathbf{u}_i$ in the remaining terms:

$$\mathbf{x}_n - \tilde{\mathbf{x}}_n = \sum_{i=M+1}^{D} \left\{ (\mathbf{x}_n - \bar{\mathbf{x}})^T \mathbf{u}_i \right\} \mathbf{u}_i$$

The reconstruction error lies entirely in the subspace spanned by the *discarded* basis vectors. Here is the geometric intuition for why this must be the case. Think of the principal subspace as a flat plane embedded in $\mathbb{R}^D$. Any point in $\mathbb{R}^D$ can be decomposed into two parts: its component *within* the plane, and its component *perpendicular* to the plane. The approximation $\tilde{\mathbf{x}}_n$ is constrained to lie on the plane — but within the plane, we have complete freedom: the z_{ni} are free parameters we can tune independently for each data point. We used that freedom optimally by setting $z_{ni} = \mathbf{x}_n^T \mathbf{u}_i$, which places $\tilde{\mathbf{x}}_n$ at exactly the right in-plane position so that the in-plane error is zero. The only remaining error is the part of $\mathbf{x}_n$ that sticks out perpendicularly from the plane — the component in the discarded directions — which we have no freedom to correct because the b_i are shared constants, not per-point variables. The minimum error is therefore always the perpendicular distance from $\mathbf{x}_n$ to the principal subspace, and the error vector always points orthogonally away from it.

3.3 Optimizing the Basis Directions

After substituting the optimal coordinates, J depends only on the choice of basis directions. The squared norm of the displacement vector is:

$$\|\mathbf{x}_n - \tilde{\mathbf{x}}_n\|^2 = \left\| \sum_{i=M+1}^{D} \{(\mathbf{x}_n - \bar{\mathbf{x}})^T \mathbf{u}_i\} \mathbf{u}_i \right\|^2 = \sum_{i=M+1}^{D} \sum_{j=M+1}^{D} \{(\mathbf{x}_n - \bar{\mathbf{x}})^T \mathbf{u}_i\} \{(\mathbf{x}_n - \bar{\mathbf{x}})^T \mathbf{u}_j\} \underbrace{\mathbf{u}_i^T \mathbf{u}_j}_{=\delta_{ij}}$$

The orthonormality condition $\mathbf{u}_i^T \mathbf{u}_j = \delta_{ij}$ kills all cross-terms ($i \neq j$), leaving only diagonal terms:

$$= \sum_{i=M+1}^{D} \left[(\mathbf{x}_n - \bar{\mathbf{x}})^T \mathbf{u}_i \right]^2$$

Rewriting each squared scalar as a quadratic form (the same trick as in Section 2.1):

$$= \sum_{i=M+1}^{D} \mathbf{u}_i^T (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T \mathbf{u}_i$$

Summing over n, dividing by N, and recognising $\mathbf{S} = \frac{1}{N} \sum_n (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T$:

$$J = \sum_{i=M+1}^{D} \mathbf{u}_i^T \mathbf{S} \mathbf{u}_i$$

We want to minimize this over the discarded directions $\{\mathbf{u}_i\}_{i=M+1}^{D}$, subject to orthonormality. Introducing Lagrange multipliers for each constraint $\mathbf{u}_i^T \mathbf{u}_i = 1$ and the off-diagonal constraints $\mathbf{u}_i^T \mathbf{u}_j = 0$ ($i \neq j$), the stationarity condition again yields the eigenvector equation $\mathbf{S}\mathbf{u}_i = \lambda_i \mathbf{u}_i$. Back-substituting $\mathbf{u}_i^T \mathbf{S} \mathbf{u}_i = \lambda_i$:

$$J = \sum_{i=M+1}^{D} \lambda_i$$

To *minimize* J, we choose the discarded directions to be the eigenvectors with the *smallest* eigenvalues.

3.4 Why Both Formulations Agree

Both formulations yield exactly the same principal subspace: the span of the M eigenvectors with the *largest* eigenvalues.

- Maximum variance: keep the M directions with the most variance (the M largest eigenvalues).
- Minimum error: discard the $D - M$ directions with the least variance (the $D - M$ smallest eigenvalues).

These are two sides of the same coin. The total variance of the data equals $\sum_{i=1}^{D} \lambda_i$ (the trace of $\mathbf{S}$). Keeping the top M eigenvalues to maximize retained variance is identical to discarding the bottom $D - M$ to minimize lost variance.

So what? The reconstruction error $J = \sum_{i=M+1}^{D} \lambda_i$ is a direct, interpretable diagnostic: it tells us exactly how much information we discard at each dimensionality.

To make this concrete: suppose your data lives in $D = 3$ dimensions but you suspect near-planar structure ($M = 2$). After running PCA you find eigenvalues $\lambda_1 = 5.0$, $\lambda_2 = 4.8$, $\lambda_3 = 0.02$. The reconstruction error from projecting to $M = 2$ is $J = 0.02$ — just 0.2% of the total variance 9.82. You can safely discard the third dimension. If instead the eigenvalues were $\lambda_1 = 3.5$, $\lambda_2 = 3.3$, $\lambda_3 = 2.9$, projecting to 2D loses $J = 2.9$ out of 9.7 — nearly 30% of the variance — and three dimensions are genuinely necessary. The eigenvalue spectrum is the readout: a sharp drop signals a natural cutoff; a flat spectrum signals that the data has no preferred low-dimensional structure.

4. Applications of PCA

4.1 Data Compression and Visualization

Once we have the principal components $\mathbf{u}_1, \ldots, \mathbf{u}_M$, the PCA approximation to any data point is:

$$\tilde{\mathbf{x}}_n = \bar{\mathbf{x}} + \sum_{i=1}^{M} \left(\mathbf{x}_n^T \mathbf{u}_i - \bar{\mathbf{x}}^T \mathbf{u}_i \right) \mathbf{u}_i$$

which reads as: "start at the sample mean, then move along each principal direction by the amount that $\mathbf{x}_n$ deviates from the mean in that direction." The compressed representation stores only the M coordinates $z_{ni} = \mathbf{x}_n^T \mathbf{u}_i - \bar{\mathbf{x}}^T \mathbf{u}_i$ per data point, replacing a D-dimensional vector with an M-dimensional one.

For visualization, we choose $M = 2$ and plot each point at $(\mathbf{x}_n^T \mathbf{u}_1, \mathbf{x}_n^T \mathbf{u}_2)$ — a scatterplot in the plane of maximum variance.

4.2 Whitening (Sphereing)

PCA can be used not for dimensionality reduction but for *normalization*. **Whitening** (also called sphereing) transforms data to have zero mean and identity covariance — decorrelating variables and placing them on a common scale. This is a prerequisite for many algorithms that assume spherical distributions, and it is standard preprocessing for ICA (see Section 7.5).

The whitening transform. Write the eigendecomposition of $\mathbf{S}$ in matrix form: $\mathbf{S} = \mathbf{U}\mathbf{L}\mathbf{U}^T$, where $\mathbf{U}$ is the $D \times D$ orthogonal matrix whose columns are eigenvectors, and $\mathbf{L} = \mathrm{diag}(\lambda_1, \ldots, \lambda_D)$ is diagonal. Define:

$$\mathbf{y}_n = \mathbf{L}^{-1/2} \mathbf{U}^T (\mathbf{x}_n - \bar{\mathbf{x}})$$

where $\mathbf{L}^{-1/2} = \mathrm{diag}(\lambda_1^{-1/2}, \ldots, \lambda_D^{-1/2})$ is the entry-wise inverse square root.

Verifying the covariance. The sample covariance of $\{\mathbf{y}_n\}$ is:

$$\frac{1}{N}\sum_{n=1}^{N}\mathbf{y}_n\mathbf{y}_n^T = \mathbf{L}^{-1/2}\mathbf{U}^T\left(\frac{1}{N}\sum_{n=1}^{N}(\mathbf{x}_n-\bar{\mathbf{x}})(\mathbf{x}_n-\bar{\mathbf{x}})^T\right)\mathbf{U}\mathbf{L}^{-1/2}$$

Substituting $\mathbf{S} = \mathbf{U}\mathbf{L}\mathbf{U}^T$:

$$= \mathbf{L}^{-1/2}\mathbf{U}^T(\mathbf{U}\mathbf{L}\mathbf{U}^T)\mathbf{U}\mathbf{L}^{-1/2} = \mathbf{L}^{-1/2}\underbrace{(\mathbf{U}^T\mathbf{U})}_{=\mathbf{I}}\mathbf{L}\underbrace{(\mathbf{U}^T\mathbf{U})}_{=\mathbf{I}}\mathbf{L}^{-1/2} = \mathbf{L}^{-1/2}\mathbf{L}\mathbf{L}^{-1/2} = \mathbf{I}$$

The whitened data has identity covariance. The zero mean follows from $\mathbb{E}[\mathbf{y}_n] = \mathbf{L}^{-1/2}\mathbf{U}^T\mathbb{E}[\mathbf{x}_n-\bar{\mathbf{x}}] = \mathbf{0}$.

Intuition. The transform does two things sequentially: $\mathbf{U}^T(\cdot)$ rotates the data into the eigenvector coordinate system, making the covariance diagonal (variables become uncorrelated); then $\mathbf{L}^{-1/2}$ rescales each axis to unit variance. The result is a spherical cloud of data — identical variance in every direction.

A caution. Near-zero eigenvalues lead to near-infinite scaling $\lambda_i^{-1/2}$, amplifying directions of negligible variance. In practice, eigenvalues below a threshold are typically discarded before whitening — this combines dimensionality reduction and normalization in a single step.

The dual trick (high-dimensional data). When $D \gg N$ (e.g., few images of millions of pixels), computing the full $D \times D$ covariance matrix costs $O(ND^2)$ and its eigenvectors cost $O(D^3)$. Since at most $N-1$ eigenvalues are nonzero (a set of N points cannot vary independently in more than $N-1$ directions), one can instead work with the $N \times N$ matrix $\frac{1}{N}\tilde{\mathbf{X}}\tilde{\mathbf{X}}^T$ (where $\tilde{\mathbf{X}}$ is the centered data matrix), which has the same nonzero eigenvalues and can be solved in $O(N^3)$. The D-dimensional eigenvectors are recovered by a single matrix-vector multiplication. This is worth knowing exists; the derivation is in Bishop §12.1.4.

5. Probabilistic PCA

5.1 Motivation and the PPCA-to-VAE Arc

Standard PCA is a deterministic algorithm: given data, compute eigenvectors. It has no notion of probability, uncertainty, or missing observations. By recasting PCA as a probabilistic generative model — Probabilistic PCA, or PPCA — we gain a likelihood function, a proper density over observations, a natural EM algorithm, and a clean handle on the structure that modern deep generative models extend.

Why PPCA matters for modern ML. PPCA is the simplest member of a family: *linear-Gaussian latent variable models*. Its generative structure is a Gaussian prior over a latent variable $\mathbf{z}$, a linear map from $\mathbf{z}$ to the data mean, and Gaussian noise. Because everything is linear and Gaussian, every relevant quantity — the marginal $p(\mathbf{x})$, the posterior $p(\mathbf{z}|\mathbf{x})$, the ELBO — has a closed-form expression. This is the *tractable* baseline.

The Variational Autoencoder (VAE) is exactly PPCA with the linear decoder $\mathbf{W}$ replaced by a neural network f_θ. That single change breaks linearity and makes the posterior intractable, forcing us to use the ELBO and an approximate encoder — all the machinery derived in the EM algorithm notes. The prior, the ELBO structure, and the "encode-to-latent, decode-to-data" frame are *directly inherited* from PPCA. Understanding PPCA makes the VAE legible rather than mysterious.

5.2 The Generative Model

The core design question is: how do we build a probability distribution over $\mathbf{x} \in \mathbb{R}^D$ that bakes in the assumption of low-dimensional structure? The answer is to write down an explicit generative process — a recipe for how observations could have been produced — and then read off the implied distributions.

Starting from the generative equation. We want every observation to be (approximately) a point on an M-dimensional linear subspace plus some noise. The most direct way to express this is:

$$\mathbf{x} = \mathbf{W}\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}$$

where:

- $\mathbf{W} \in \mathbb{R}^{D\times M}$ is the *loading matrix* — its columns span the principal subspace. Multiplying the M-dimensional code $\mathbf{z}$ by $\mathbf{W}$ maps it into a point on the subspace inside $\mathbb{R}^D$.
- $\mathbf{z} \in \mathbb{R}^M$ is the latent code — a low-dimensional vector specifying *where on the subspace* the observation lives.
- $\boldsymbol{\mu} \in \mathbb{R}^D$ is the overall data mean, shifting the subspace to the right location in $\mathbb{R}^D$.

- $\boldsymbol{\epsilon} \in \mathbb{R}^D$ is noise capturing off-subspace variation.

This is the entire model. Everything else follows from completing the specification of $\mathbf{z}$ and $\boldsymbol{\epsilon}$.

Choosing the noise distribution. We take $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$ — *isotropic* Gaussian noise (*isotropic* meaning variance σ^2 is equal in every direction, enforced by the scalar multiple of the identity). This is a deliberate simplification: it says all directions perpendicular to the principal subspace are equally noisy. Section 5.7 discusses Factor Analysis, which relaxes this by giving each dimension its own noise variance. The payoff for the isotropic assumption is tractability — the marginal $p(\mathbf{x})$ and the posterior $p(\mathbf{z}|\mathbf{x})$ both have closed forms.

Deriving $p(\mathbf{x} \mid \mathbf{z})$ from the generative equation. Given a fixed value of $\mathbf{z}$, the generative equation becomes:

$$\mathbf{x} = \underbrace{(\mathbf{W}\mathbf{z} + \boldsymbol{\mu})}_{\text{fixed vector}} + \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$$

We are adding a fixed vector to a Gaussian random vector. Adding a constant to a Gaussian shifts its mean and leaves its covariance unchanged:

$$p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x} \mid \mathbf{W}\mathbf{z} + \boldsymbol{\mu},\ \sigma^2\mathbf{I}), \qquad \mathbf{x} \in \mathbb{R}^D$$

Choosing the prior on $\mathbf{z}$: why $\mathcal{N}(\mathbf{0}, \mathbf{I})$ without loss of generality. We place a standard isotropic Gaussian prior over the latent code:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z} \mid \mathbf{0}, \mathbf{I}), \qquad \mathbf{z} \in \mathbb{R}^M$$

This might seem restrictive. Why not a general Gaussian $\mathcal{N}(\mathbf{m}, \boldsymbol{\Sigma})$ with arbitrary mean and covariance? Because such a model is no more expressive — any general Gaussian prior is equivalent to a standard Gaussian prior with reparameterized loading matrix and mean. To see this, suppose we used $p(\mathbf{z}) = \mathcal{N}(\mathbf{m}, \boldsymbol{\Sigma})$. Introduce the change of variables $\mathbf{z}' = \boldsymbol{\Sigma}^{-1/2}(\mathbf{z} - \mathbf{m})$, so that $\mathbf{z} = \boldsymbol{\Sigma}^{1/2}\mathbf{z}' + \mathbf{m}$ and $p(\mathbf{z}') = \mathcal{N}(\mathbf{0}, \mathbf{I})$ by construction. Substituting into the generative equation:

$$\mathbf{x} = \mathbf{W}\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon} = \mathbf{W}(\boldsymbol{\Sigma}^{1/2}\mathbf{z}' + \mathbf{m}) + \boldsymbol{\mu} + \boldsymbol{\epsilon} = \underbrace{(\mathbf{W}\boldsymbol{\Sigma}^{1/2})}_{\mathbf{W}'}\mathbf{z}' + \underbrace{(\mathbf{W}\mathbf{m} + \boldsymbol{\mu})}_{\boldsymbol{\mu}'} + \boldsymbol{\epsilon}$$

The general-prior model with parameters $(\mathbf{W}, \boldsymbol{\mu}, \mathbf{m}, \boldsymbol{\Sigma})$ is identical to the standard-prior model with parameters $(\mathbf{W}', \boldsymbol{\mu}')$. The extra degrees of freedom $\mathbf{m}$ and $\boldsymbol{\Sigma}$ are completely absorbed into $\mathbf{W}'$ and $\boldsymbol{\mu}'$ — no data can ever distinguish the two. Fixing $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ is genuinely without loss of generality.

Summary: the PPCA model. Putting both pieces together:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z} \mid \mathbf{0}, \mathbf{I})$$

$$p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x} \mid \mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \sigma^2\mathbf{I})$$

The generative process. To generate one data point:

1. Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ — pick a location in latent space
2. Compute the noiseless point $\mathbf{W}\mathbf{z} + \boldsymbol{\mu}$ — map it to the subspace in $\mathbb{R}^D$
3. Add noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$ — perturb off the subspace

572 12. CONTINUOUS LATENT VARIABLES

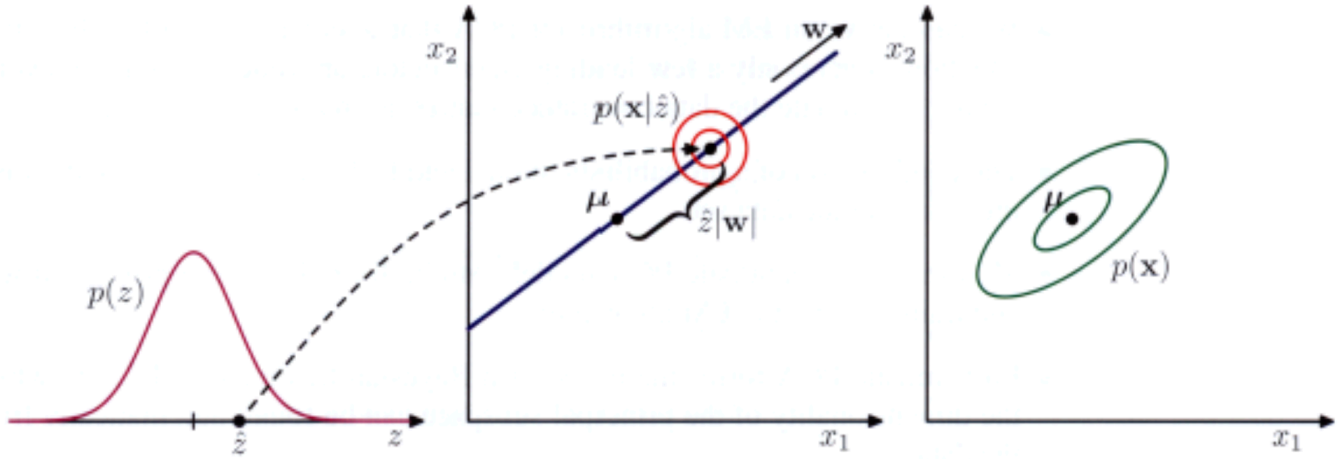

Figure 12.9 An illustration of the generative view of the probabilistic PCA model for a two-dimensional data space and a one-dimensional latent space. An observed data point $\mathbf{x}$ is generated by first drawing a value $\hat{z}$ for the latent variable from its prior distribution $p(z)$ and then drawing a value for $\mathbf{x}$ from an isotropic Gaussian distribution (illustrated by the red circles) having mean $\mathbf{w}\hat{z} + \boldsymbol{\mu}$ and covariance $\sigma^2\mathbf{I}$. The green ellipses show the density contours for the marginal distribution $p(\mathbf{x})$.

Intuition. The model says every observation is a noisy version of a point on the M-dimensional principal subspace. The subspace is defined by $\mathbf{W}$; the location along the subspace is determined by $\mathbf{z}$; and σ^2 controls how far off-subspace observations can be. **When $\sigma^2 \to 0$, observations lie exactly on the subspace and we recover standard PCA.**

5.3 The Marginal Distribution $p(\mathbf{x})$

To fit the model, we need $p(\mathbf{x})$ — the marginal over observed data, obtained by integrating out the latent variable:

$$p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z})p(\mathbf{z})\, d\mathbf{z}$$

Since $\mathbf{x} = \mathbf{W}\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}$ is a linear function of two independent Gaussians, the marginal is also Gaussian. We find it by computing the mean and covariance of $\mathbf{x}$ directly.

Mean of $\mathbf{x}$:

$$\mathbb{E}[\mathbf{x}] = \mathbb{E}[\mathbf{W}\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}] = \mathbf{W}\underbrace{\mathbb{E}[\mathbf{z}]}_{=\mathbf{0}} + \boldsymbol{\mu} + \underbrace{\mathbb{E}[\boldsymbol{\epsilon}]}_{=\mathbf{0}} = \boldsymbol{\mu}$$

Covariance of $\mathbf{x}$. The covariance is $\mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T] = \mathbb{E}[(\mathbf{W}\mathbf{z} + \boldsymbol{\epsilon})(\mathbf{W}\mathbf{z} + \boldsymbol{\epsilon})^T]$. Expanding:

$$= \mathbb{E}[\mathbf{W}\mathbf{z}\mathbf{z}^T\mathbf{W}^T] + \mathbb{E}[\mathbf{W}\mathbf{z}\boldsymbol{\epsilon}^T] + \mathbb{E}[\boldsymbol{\epsilon}\mathbf{z}^T\mathbf{W}^T] + \mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^T]$$

Since $\mathbf{z}$ and $\boldsymbol{\epsilon}$ are independent with zero means, the cross terms $\mathbb{E}[\mathbf{z}\boldsymbol{\epsilon}^T] = \mathbb{E}[\mathbf{z}]\mathbb{E}[\boldsymbol{\epsilon}]^T = \mathbf{0}$ vanish. Therefore:

$$= \mathbf{W}\underbrace{\mathbb{E}[\mathbf{z}\mathbf{z}^T]}_{=\mathbf{I}}\mathbf{W}^T + \underbrace{\mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^T]}_{=\sigma^2\mathbf{I}} = \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I}$$

The two expectations follow directly from the model distributions. For any random vector, the covariance is defined as $\mathbb{E}[(\mathbf{z} - \boldsymbol{\mu})(\mathbf{z} - \boldsymbol{\mu})^T]$. Expanding the outer product and using linearity of expectation gives $\mathbb{E}[\mathbf{z}\mathbf{z}^T] - \boldsymbol{\mu}\boldsymbol{\mu}^T$. Since $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ has mean $\boldsymbol{\mu} = \mathbf{0}$, the second term vanishes and the covariance equals $\mathbb{E}[\mathbf{z}\mathbf{z}^T]$ directly. Setting this equal to the known covariance matrix $\mathbf{I}$ gives $\mathbb{E}[\mathbf{z}\mathbf{z}^T] = \mathbf{I}$. The same argument applied to $p(\boldsymbol{\epsilon}) = \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$ gives $\mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^T] = \sigma^2\mathbf{I}$.

Therefore:

$$\boxed{p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \mathbf{C}), \qquad \mathbf{C} = \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I}}$$

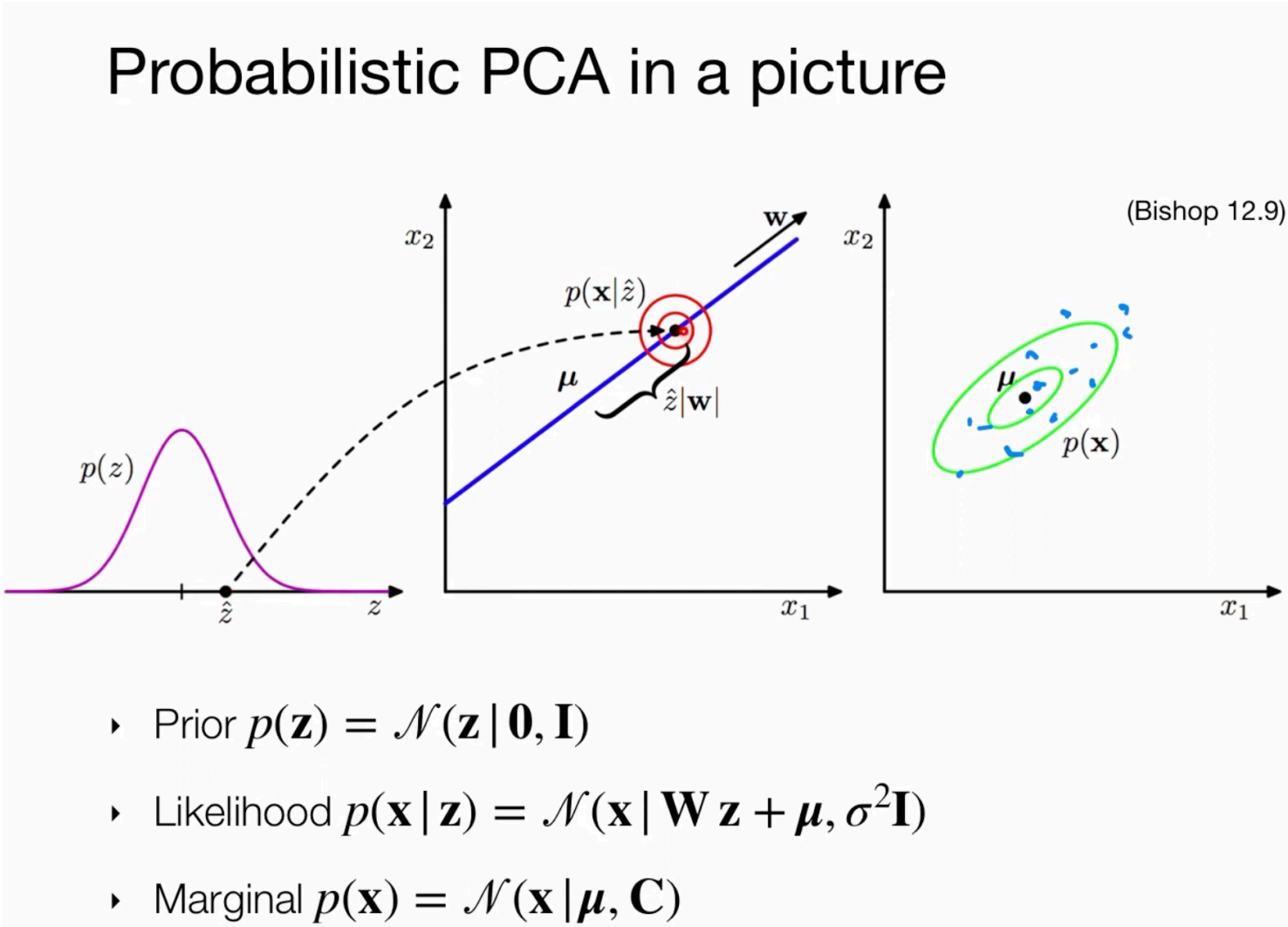

Image source: https://www.youtube.com/watch?v=invkqcdSkco&t=938s

Unpacking $\mathbf{C}$. The structured form $\mathbf{C} = \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I}$ encodes a precise geometric picture. To unpack it, compute the variance of the data in an arbitrary unit direction $\mathbf{v}$. Since $\mathbb{E}[\mathbf{x}] = \boldsymbol{\mu}$, the scalar projection $\mathbf{v}^T\mathbf{x}$ has mean $\mathbf{v}^T\boldsymbol{\mu}$, so:

$$\mathrm{Var}(\mathbf{v}^T\mathbf{x}) = \mathbb{E}\big[(\mathbf{v}^T\mathbf{x} - \mathbf{v}^T\boldsymbol{\mu})^2\big] = \mathbb{E}\big[(\mathbf{v}^T(\mathbf{x} - \boldsymbol{\mu}))^2\big]$$

The squared scalar $(\mathbf{v}^T\mathbf{a})^2$ can be rewritten as the quadratic form $\mathbf{v}^T\mathbf{a}\mathbf{a}^T\mathbf{v}$ (the same identity used in Section 2.1). Applying that here with $\mathbf{a} = \mathbf{x} - \boldsymbol{\mu}$ and pulling the constant $\mathbf{v}$ outside the expectation:

$$= \mathbb{E}\big[\mathbf{v}^T(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T\mathbf{v}\big] = \mathbf{v}^T\underbrace{\mathbb{E}\big[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T\big]}_{=\mathbf{C}}\mathbf{v} = \mathbf{v}^T\mathbf{C}\mathbf{v}$$

Substituting $\mathbf{C} = \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I}$ and using $\mathbf{v}^T\mathbf{v} = 1$:

$$\mathrm{Var}(\mathbf{v}^T\mathbf{x}) = \mathbf{v}^T\mathbf{W}\mathbf{W}^T\mathbf{v} + \sigma^2\underbrace{\mathbf{v}^T\mathbf{v}}_{=1} = \|\mathbf{W}^T\mathbf{v}\|^2 + \sigma^2$$

where we used the identity $\mathbf{v}^T(\mathbf{A}^T\mathbf{A})\mathbf{v} = \|\mathbf{A}\mathbf{v}\|^2$ with $\mathbf{A} = \mathbf{W}^T$. Now the two cases are explicit:

- **$\mathbf{v}$ lies in the principal subspace** (the column space of $\mathbf{W}$): $\mathbf{W}^T\mathbf{v}$ is the coordinate representation of $\mathbf{v}$ in the M-dimensional latent space, and $\|\mathbf{W}^T\mathbf{v}\|^2 > 0$. The total variance is *signal* ($\|\mathbf{W}^T\mathbf{v}\|^2$) plus *noise* ($\sigma^2$).
- **$\mathbf{v}$ is orthogonal to the principal subspace**: $\mathbf{v}$ is perpendicular to every column of $\mathbf{W}$, so $\mathbf{W}^T\mathbf{v} = \mathbf{0}$ and $\|\mathbf{W}^T\mathbf{v}\|^2 = 0$. The total variance collapses to just σ^2 — pure noise, no signal whatsoever.

This is the precise version of the "pancake" intuition: in the M principal directions the distribution is spread out by the signal plus noise; in all $D - M$ orthogonal directions it is uniformly thin, with spread controlled only by σ^2. The distribution concentrates on the principal subspace, and as $\sigma^2 \to 0$ it collapses onto it entirely — recovering standard PCA.

Also worth noting: $\mathbf{W}\mathbf{W}^T$ is rank M (not full rank D), because $\mathbf{W}$ is $D \times M$ with $M < D$ — it maps $\mathbb{R}^M$ into $\mathbb{R}^D$ and can span at most M directions. The $\sigma^2\mathbf{I}$ term is what fills in the remaining $D - M$ directions with nonzero variance, making $\mathbf{C}$ full rank and the Gaussian non-degenerate.

So what? It is worth pausing to ask why we computed $p(\mathbf{x})$ at all. After all, the generative process only requires $p(\mathbf{z})$ and $p(\mathbf{x}|\mathbf{z})$ — sample a latent code, then sample an observation. That is the *forward* direction: latent $\rightarrow$ data.

But a model is only useful if we can also run *backward*: data $\rightarrow$ parameters and latent codes. That backward direction requires $p(\mathbf{x})$ in two ways:

- **Fitting the model.** To estimate $\mathbf{W}$ and σ^2 from data, we maximize the likelihood of the observations. That likelihood *is* $p(\mathbf{x})$ — there is no other way to write down $\log p(\mathbf{X} \mid \mathbf{W}, \sigma^2)$. This is exactly what Section 5.4 does.
- **Mapping observations back to latent codes.** By Bayes' theorem, $p(\mathbf{z}|\mathbf{x}) \propto p(\mathbf{x}|\mathbf{z})p(\mathbf{z})$, with $p(\mathbf{x})$ as the normalizing constant. Section 5.5 derives this posterior in closed form.

As a bonus, $p(\mathbf{x})$ lets you assign an actual probability to any observation — useful for anomaly detection and model comparison. Standard PCA can only tell you the distance from the subspace; PPCA tells you the full probability under the model. Computing $p(\mathbf{x})$ is what converts the generative story into a proper probabilistic model that can be fit, evaluated, and queried.

5.4 Maximum Likelihood Solution

Setting up the log-likelihood. Since $p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \mathbf{C})$ and observations are i.i.d., the log-likelihood is a sum of individual log-densities. The log-density of a single multivariate Gaussian is:

$$\ln \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}, \mathbf{C}) = -\frac{D}{2}\ln(2\pi) - \frac{1}{2}\ln|\mathbf{C}| - \frac{1}{2}(\mathbf{x}_n - \boldsymbol{\mu})^T\mathbf{C}^{-1}(\mathbf{x}_n - \boldsymbol{\mu})$$

Summing over n, the first two terms scale trivially to $-\frac{N}{2}(D\ln(2\pi) + \ln|\mathbf{C}|)$. The quadratic terms require more care. Each $(\mathbf{x}_n - \boldsymbol{\mu})^T\mathbf{C}^{-1}(\mathbf{x}_n - \boldsymbol{\mu})$ is a scalar, so we can apply the **trace trick**: for any scalar $a = \mathbf{v}^T\mathbf{A}\mathbf{v}$, we have $a = \mathrm{Tr}(\mathbf{v}^T\mathbf{A}\mathbf{v}) = \mathrm{Tr}(\mathbf{A}\mathbf{v}\mathbf{v}^T)$, where the second equality uses the cyclic property of the trace ($\mathrm{Tr}(\mathbf{ABC}) = \mathrm{Tr}(\mathbf{CAB})$). Applying this with $\mathbf{v} = \mathbf{x}_n - \boldsymbol{\mu}$ and $\mathbf{A} = \mathbf{C}^{-1}$:

$$\sum_{n=1}^{N}(\mathbf{x}_n - \boldsymbol{\mu})^T\mathbf{C}^{-1}(\mathbf{x}_n - \boldsymbol{\mu}) = \sum_{n=1}^{N}\mathrm{Tr}\big(\mathbf{C}^{-1}(\mathbf{x}_n - \boldsymbol{\mu})(\mathbf{x}_n - \boldsymbol{\mu})^T\big) = \mathrm{Tr}\left(\mathbf{C}^{-1}\sum_{n=1}^{N}(\mathbf{x}_n - \boldsymbol{\mu})(\mathbf{x}_n - \boldsymbol{\mu})^T\right)$$

where the last step pulls $\mathbf{C}^{-1}$ (constant in n) outside the sum using linearity of the trace. Dividing by N inside the trace recognizes the sample covariance $\mathbf{S}_\mu = \frac{1}{N}\sum_n(\mathbf{x}_n - \boldsymbol{\mu})(\mathbf{x}_n - \boldsymbol{\mu})^T$, giving a total contribution of $N\,\mathrm{Tr}(\mathbf{C}^{-1}\mathbf{S}_\mu)$. Combining all terms:

$$\ln p(\mathbf{X} \mid \boldsymbol{\mu}, \mathbf{W}, \sigma^2) = -\frac{N}{2}\left\{D\ln(2\pi) + \ln|\mathbf{C}| + \mathrm{Tr}\left(\mathbf{C}^{-1}\mathbf{S}_\mu\right)\right\}$$

Solving for $\boldsymbol{\mu}$. Since $\mathbf{C}$ does not depend on $\boldsymbol{\mu}$, maximizing over $\boldsymbol{\mu}$ reduces to minimizing $\mathrm{Tr}(\mathbf{C}^{-1}\mathbf{S}_\mu)$. This is a standard least-squares argument: $\mathbf{S}_\mu$ is minimized when $\boldsymbol{\mu} = \bar{\mathbf{x}}$ (the sample mean is the point that minimizes total squared distance to the data). Substituting $\boldsymbol{\mu}_{\mathrm{ML}} = \bar{\mathbf{x}}$ replaces $\mathbf{S}_\mu$ with $\mathbf{S}$:

$$\ln p(\mathbf{X} \mid \mathbf{W}, \sigma^2) = -\frac{N}{2}\left\{D\ln(2\pi) + \ln|\mathbf{C}| + \mathrm{Tr}\left(\mathbf{C}^{-1}\mathbf{S}\right)\right\}$$

The closed-form solution for $\mathbf{W}$ and σ^2. Maximizing over $\mathbf{W}$ and σ^2 is non-trivial — it requires the matrix determinant lemma and the Woodbury identity to handle $|\mathbf{C}|$ and $\mathbf{C}^{-1}$ analytically. The full derivation is in Tipping and Bishop (1999); the result is:

$$\mathbf{W}_{\mathrm{ML}} = \mathbf{U}_M(\mathbf{L}_M - \sigma^2_{\mathrm{ML}}\mathbf{I})^{1/2}\mathbf{R}$$

where:

- $\mathbf{U}_M \in \mathbb{R}^{D\times M}$: columns are the M eigenvectors of $\mathbf{S}$ with the M largest eigenvalues.
- $\mathbf{L}_M = \mathrm{diag}(\lambda_1, \ldots, \lambda_M)$: diagonal matrix of those eigenvalues.
- $\mathbf{R} \in \mathbb{R}^{M\times M}$: an *arbitrary* orthogonal matrix, reflecting rotational non-identifiability (discussed below).

Reading the $\mathbf{W}_{\mathrm{ML}}$ formula. Setting $\mathbf{R} = \mathbf{I}$ for clarity, the ith column of $\mathbf{W}_{\mathrm{ML}}$ is $\sqrt{\lambda_i - \sigma^2_{\mathrm{ML}}}\,\mathbf{u}_i$. The scaling factor $\sqrt{\lambda_i - \sigma^2}$ is the key: λ_i is the *total* observed variance in direction $\mathbf{u}_i$, but PPCA attributes σ^2 of that to noise. The remainder $\lambda_i - \sigma^2$ is the *signal* variance — the portion genuinely explained by the latent structure. The column is scaled by the signal standard deviation. As a sanity check: substituting back, the implied marginal covariance in direction $\mathbf{u}_i$ is $(\lambda_i - \sigma^2) + \sigma^2 = \lambda_i$, exactly matching the data. Directions where $\lambda_i \approx \sigma^2$ (signal barely above noise) get near-zero columns; strongly expressed directions ($\lambda_i \gg \sigma^2$) get large ones.

The corresponding optimal noise variance is:

$$\sigma^2_{\text{ML}} = \frac{1}{D-M} \sum_{i=M+1}^{D} \lambda_i$$

In the discarded directions the model has no signal ($\mathbf{W}$ has no components there), so all variance in those directions is attributed to noise. The MLE sets σ^2 to the mean of the eigenvalues it cannot explain — the average residual variance across the $D - M$ discarded dimensions.

The ML solution selects the top eigenvectors. All other subsets of M eigenvectors are saddle points, not maxima (Tipping and Bishop, 1999). This confirms that PPCA recovers exactly the same principal subspace as standard PCA, but via a principled maximum-likelihood procedure.

Rotational non-identifiability. The matrix $\mathbf{R}$ is arbitrary because the model is invariant to any rotation of the latent space. Replacing $\mathbf{W}$ with $\tilde{\mathbf{W}} = \mathbf{W}\mathbf{R}$ and $\mathbf{z}$ with $\tilde{\mathbf{z}} = \mathbf{R}^T\mathbf{z}$ leaves $\mathbf{W}\mathbf{z} = \tilde{\mathbf{W}}\tilde{\mathbf{z}}$ unchanged. Since $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ is spherically symmetric (invariant to rotations), the prior on $\tilde{\mathbf{z}}$ is also $\mathcal{N}(\mathbf{0}, \mathbf{I})$, and the entire model is identical. No data can distinguish between $\mathbf{W}$ and $\mathbf{W}\mathbf{R}$ for any orthogonal $\mathbf{R}$.

This is the same non-identifiability that afflicts Factor Analysis (see Section 5.7) and is the reason ICA is needed when we want interpretable individual components rather than just the subspace.

5.5 The Posterior $p(\mathbf{z} \mid \mathbf{x})$ and the Latent Projection

For visualization and compression, we want to map observations *back* to the latent space. Since PPCA is a linear-Gaussian model, the posterior is Gaussian with a closed-form expression (derived from the general linear-Gaussian result; see Bishop §2.3):

$$p(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\Big(\mathbf{z} \;\Big|\; \mathbf{M}^{-1}\mathbf{W}^T(\mathbf{x} - \boldsymbol{\mu}),\ \sigma^2\mathbf{M}^{-1}\Big)$$

where $\mathbf{M} = \mathbf{W}^T\mathbf{W} + \sigma^2\mathbf{I} \in \mathbb{R}^{M \times M}$ is an $M \times M$ matrix (small and easily invertible).

Reading the posterior mean. The expression $\mathbf{M}^{-1}\mathbf{W}^T(\mathbf{x} - \boldsymbol{\mu})$ computes the best estimate of the latent code in two steps. First, $\mathbf{W}^T(\mathbf{x} - \boldsymbol{\mu})$ projects the centered observation into the M-dimensional latent space: each entry of the resulting vector is the inner product of $(\mathbf{x} - \boldsymbol{\mu})$ with the corresponding column of $\mathbf{W}$, measuring how strongly that principal direction is expressed in the data. Second, $\mathbf{M}^{-1}$ corrects this raw projection: $\mathbf{M} = \mathbf{W}^T\mathbf{W} + \sigma^2\mathbf{I}$ accounts for both the self-overlap of $\mathbf{W}$'s columns ($\mathbf{W}^T\mathbf{W}$) and the noise ($\sigma^2\mathbf{I}$), preventing the estimate from over-committing when the signal is noisy.

Why the posterior covariance $\sigma^2\mathbf{M}^{-1}$ is the same for every observation. This seems counterintuitive — shouldn't we be more uncertain about some data points than others? The key is that the uncertainty about $\mathbf{z}$ given $\mathbf{x}$ comes entirely from the noise σ^2: multiple latent codes $\mathbf{z}$ could have plausibly generated any particular $\mathbf{x}$ through the noise channel, and the *extent* of that ambiguity depends only on how noisy the channel is (σ^2) and how the loading matrix covers the latent space ($\mathbf{M}^{-1}$). Neither of these changes with the specific observation. Think of it like GPS uncertainty: the precision of your position fix depends on the receiver hardware and satellite geometry — not on where you actually are.

Shrinkage toward the origin. For $\sigma^2 > 0$, the posterior mean is pulled toward $\mathbf{0}$ relative to the raw projection. This is a Bayesian regularization effect: the prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ asserts that latent codes should cluster near the origin, and the posterior balances this prior belief against the data evidence. The larger σ^2, the less we trust the data and the more we shrink toward the prior mean. The $\sigma^2\mathbf{I}$ term in $\mathbf{M}$ is what produces the shrinkage — it inflates the denominator, pulling the estimate back.

Connection to standard PCA. In the limit $\sigma^2 \to 0$, $\mathbf{M} \to \mathbf{W}^T\mathbf{W}$ and the shrinkage vanishes. The posterior mean becomes $(\mathbf{W}^T\mathbf{W})^{-1}\mathbf{W}^T(\mathbf{x} - \boldsymbol{\mu})$, which is the Moore-Penrose pseudoinverse of $\mathbf{W}$ applied to $(\mathbf{x} - \boldsymbol{\mu})$ — equivalently, the least-squares solution to $\mathbf{W}\mathbf{z} \approx (\mathbf{x} - \boldsymbol{\mu})$, i.e., the orthogonal projection onto the column space of $\mathbf{W}$. This is exactly standard PCA. The posterior covariance $\sigma^2\mathbf{M}^{-1} \to \mathbf{0}$, collapsing all probability mass onto the subspace.

So what? The PPCA posterior is the closed-form version of what the VAE encoder approximates. In PPCA, $p(\mathbf{z}|\mathbf{x})$ is a tractable Gaussian because the model is linear-Gaussian. In the VAE, the decoder f_θ is a neural network, making $p(\mathbf{z}|\mathbf{x})$ intractable — the encoder $q_\phi(\mathbf{z}|\mathbf{x})$ is trained to approximate it, and the ELBO is maximized rather than the exact likelihood. The PPCA posterior is the anchor that makes this connection precise.

5.6 EM for PPCA (Pointer)

PPCA has a closed-form ML solution, but there are practical reasons to use EM instead: it avoids forming the $D \times D$ covariance matrix explicitly, handles missing data naturally, and extends to models like Factor Analysis where no closed form exists.

The EM algorithm for PPCA follows the standard pattern from the EM algorithm notes. The E-step computes $\mathbb{E}[\mathbf{z}_n]$ and $\mathbb{E}[\mathbf{z}_n\mathbf{z}_n^T]$ under the current posterior; the M-step updates $\mathbf{W}$ and σ^2 by maximizing the expected complete-data log-likelihood. For the detailed derivation and equations, see the EM algorithm notes.

A useful physical analogy makes the geometric meaning of the EM steps clear. Picture the data points as masses attached by springs to a rigid rod representing the principal subspace. In the E-step, the rod is held fixed and each mass slides along the rod to minimize its spring energy — this is orthogonal projection. In the M-step, the attachment points are fixed and the rod is released to settle at the minimum-energy position — this rotates the subspace to better fit the data. Iterating converges to the ML solution.

5.7 Factor Analysis: One Structural Difference

Factor Analysis (FA) differs from PPCA in a single assumption. In PPCA, the noise covariance is $\sigma^2\mathbf{I}$ — isotropic, the same in every direction. In FA:

$$p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x} \mid \mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \boldsymbol{\Psi})$$

where $\boldsymbol{\Psi} = \mathrm{diag}(\psi_1, \ldots, \psi_D)$ is *diagonal*, allowing each observed variable its own independent noise variance. The columns of $\mathbf{W}$ are called **factor loadings** (they capture correlations between observed variables); the diagonal elements ψ_i are called **uniquenesses** (variance unique to each variable, unexplained by the shared factors).

This change has a concrete consequence for symmetry: FA is covariant under *component-wise rescaling* of the data (rescaling variable i by a_i is absorbed into rescaling ψ_i and the ith row of $\mathbf{W}$), whereas PPCA is covariant under *rotations* of the data space. FA is therefore the right tool when different variables have inherently different noise levels; PPCA when the noise is isotropic.

Both models share the rotational non-identifiability of the latent space. This is the source of historical controversy in FA: analysts have tried to "interpret" individual factors, but any rotation $\mathbf{W} \to \mathbf{W}\mathbf{R}$ produces an equivalent model. The problem is not FA-specific — it is endemic to any latent variable model with a Gaussian (rotationally symmetric) latent prior.

6. From PCA to ICA: Why Non-Gaussianity Matters

PCA and ICA form a natural hierarchy in what they can identify. PCA finds the *subspace* of maximum variance, but within that subspace cannot determine a canonical orientation — any rotation is equally valid. ICA goes further: given non-Gaussian sources, it finds the specific rotation that makes the recovered components statistically *independent*, not just uncorrelated. The tool that makes ICA possible, and PCA blind to, is non-Gaussianity.

6.1 PCA's Fundamental Limitation: Rotational Ambiguity

As established in Section 5.4, the PPCA likelihood is invariant under any rotation $\mathbf{W} \to \mathbf{W}\mathbf{R}$ of the latent space. This is not a modeling weakness that more data can resolve — it is a fundamental property of Gaussian distributions.

Why? A multivariate Gaussian $\mathcal{N}(\mathbf{0}, \mathbf{I})$ has density proportional to $\exp(-\|\mathbf{z}\|^2/2)$. This function depends only on the norm $\|\mathbf{z}\|$, not on the direction — it is *spherically symmetric*. Any rotation of $\mathbf{z}$ leaves the distribution unchanged. Consequently, the distribution of $\mathbf{W}\mathbf{z}$ is indistinguishable from $(\mathbf{W}\mathbf{R})(\mathbf{R}^T\mathbf{z})$ for any orthogonal $\mathbf{R}$: both produce a Gaussian with the same covariance $\mathbf{W}\mathbf{W}^T$.

PCA finds the subspace of maximum variance, but within that subspace it returns an arbitrary basis. The principal components are unique only when eigenvalues are distinct — and even then, the signs of the eigenvectors are undetermined.

The question this raises. Is there a *preferred* rotation — one that reveals meaningful structure? If the latent variables have non-Gaussian marginals and are *genuinely statistically independent*, then the distribution is no longer spherically symmetric and a preferred orientation exists. This is the motivation for ICA.

6.2 The Cocktail Party Problem

The canonical motivation for ICA (following Hyvärinen and Oja, 2000):

Two speakers talk simultaneously in a room. Two microphones at different positions record their voices. Each microphone captures a weighted mixture of the two speech signals. Let $s_1(t)$ and $s_2(t)$ be the speech signals (sources) and $x_1(t)$, $x_2(t)$ the microphone recordings. Then:

$$x_j(t) = \sum_k a_{jk} s_k(t), \qquad j = 1, 2$$

or in matrix form: $\mathbf{x}(t) = \mathbf{A}\mathbf{s}(t)$.

The mixing coefficients a_{jk} depend on the distances and acoustic properties of the room. Given only the recorded signals $\mathbf{x}(t)$ — without knowing $\mathbf{A}$ or the original sources — can we recover s_1 and s_2?

This is **blind source separation** ("blind" because neither the sources nor the mixing matrix are observed). ICA solves it by exploiting the statistical independence of the sources: the activity of one speaker does not provide information about the other.

6.3 The ICA Model

The model. We observe n linear mixtures of n independent components:

$$\mathbf{x} = \mathbf{A}\mathbf{s}$$

where $\mathbf{x} \in \mathbb{R}^n$ is the vector of observations, $\mathbf{s} \in \mathbb{R}^n$ is the vector of independent sources, and $\mathbf{A} \in \mathbb{R}^{n \times n}$ is the unknown *mixing matrix*, assumed square and invertible. Both $\mathbf{s}$ and $\mathbf{A}$ are unknown; we observe only $\mathbf{x}$.

Goal. Estimate the *demixing matrix* $\mathbf{W} = \mathbf{A}^{-1}$ such that $\hat{\mathbf{s}} = \mathbf{W}\mathbf{x}$ recovers the sources.

The key assumption. The components $s_1, \ldots, s_n$ are *statistically independent*: their joint density factorizes as:

$$p(\mathbf{s}) = \prod_{j=1}^{n} p_j(s_j)$$

Each source has its own marginal density p_j; knowing the value of one source gives no information about any other.

Relationship to PPCA. In PPCA, the latent variable also has a factored prior — $\mathcal{N}(\mathbf{0}, \mathbf{I}) = \prod_j \mathcal{N}(0, 1)$ — but the marginals are Gaussian. ICA allows non-Gaussian marginals and assumes a noise-free model ($\mathbf{x} = \mathbf{A}\mathbf{s}$ exactly). When the marginals are non-Gaussian, the rotational ambiguity vanishes: there is a unique mixing matrix (up to permutation and scaling) that makes the recovered components independent.

6.4 Ambiguities Inherent in ICA

Even with non-Gaussian sources, two fundamental ambiguities remain:

Scaling ambiguity. We cannot determine the variance of the independent components, because any scalar factor absorbed into s_i can be cancelled by the corresponding column of $\mathbf{A}$. *Convention*: fix the variance of each s_i to 1, i.e., $\mathbb{E}[s_i^2] = 1$. A sign ambiguity persists (s_i and $-s_i$ are indistinguishable).

Permutation ambiguity. We cannot determine the order of the components, because any permutation of the sources can be absorbed into the columns of $\mathbf{A}$. *Convention*: accept whatever ordering the algorithm produces.

These ambiguities are fundamental and unavoidable without additional prior information. In practice they are rarely consequential: the goal is usually to identify the sources and their mixing structure, not to order them in any particular way.

6.5 Why Gaussian Latents Are Forbidden

This is the central constraint of ICA, and it deserves a precise explanation.

The argument. Suppose all sources $s_1, \ldots, s_n$ are Gaussian and independent, and the mixing matrix $\mathbf{A}$ is orthogonal. Then the observations $\mathbf{x} = \mathbf{A}\mathbf{s}$ are a linear transformation of independent Gaussians. Their covariance is $\mathbf{A}\,\mathbb{E}[\mathbf{s}\mathbf{s}^T]\,\mathbf{A}^T = \mathbf{A}\mathbf{I}\mathbf{A}^T = \mathbf{I}$ (using independence and unit variance of the sources, and orthogonality of $\mathbf{A}$). So the observations are i.i.d. standard Gaussians.

The joint density is $p(\mathbf{x}) = (2\pi)^{-n/2} \exp(-\|\mathbf{x}\|^2/2)$, which is *rotationally symmetric* — it depends only on $\|\mathbf{x}\|$. For any other orthogonal matrix $\mathbf{B}$, substituting $\mathbf{A}\mathbf{B}^T$ for $\mathbf{A}$ (and $\mathbf{B}\mathbf{s}$ for $\mathbf{s}$) produces exactly the same distribution over $\mathbf{x}$, since $\mathbf{B}\mathbf{s}$ is still i.i.d. standard Gaussian. No data can ever distinguish between these infinitely many mixing matrices. The mixing matrix is not identifiable.

Intuition. A spherically symmetric distribution contains no information about "special directions" — every axis is equivalent. Non-Gaussian distributions break this symmetry: they have preferred orientations (the directions of the independent components), which ICA finds.

The practical rule. At most one source can be Gaussian for the ICA model to be identifiable. Real-world signals — speech, images, financial returns — typically have heavier tails or sharper peaks than Gaussians (supergaussian distributions), which is why ICA is so effective for them.

6.6 Independence vs. Uncorrelatedness

A crucial and commonly confused distinction:

Uncorrelated: $\mathbb{E}[s_i s_j] - \mathbb{E}[s_i]\mathbb{E}[s_j] = 0$ for $i \neq j$ (zero linear correlation). This is a *second-order* property — it constrains only pairwise covariances.

Independent: $p(s_i, s_j) = p(s_i)p(s_j)$ for all $i \neq j$. This constrains the entire joint distribution, not just second moments.

Independence implies uncorrelatedness, but the converse fails. A clean discrete counterexample: draw (y_1, y_2) uniformly from $\{(0,1),(0,-1),(1,0),(-1,0)\}$. Then $\mathbb{E}[y_1 y_2] = 0$ (uncorrelated), but knowing $y_1 = 1$ forces $y_2 = 0$ (dependent). The factorization condition is violated: $\mathbb{E}[y_1^2 y_2^2] = 0 \neq \frac{1}{4} = \mathbb{E}[y_1^2]\mathbb{E}[y_2^2]$.

Relevance to PCA vs. ICA. PCA finds an orthogonal rotation that *decorrelates* the data — after PCA, the principal components have zero covariance. But decorrelation is only a second-order condition. For non-Gaussian distributions, zero covariance does not imply statistical independence. ICA goes further: it finds the transformation that makes the components statistically independent — zero covariance *and* zero higher-order dependence. This is why ICA can find the original sources where PCA cannot: the sources are independent, not merely uncorrelated.

7. ICA Estimation: Finding Independent Components

7.1 The Core Idea: Non-Gaussianity as a Target

The Central Limit Theorem tells us that sums of independent random variables tend toward Gaussian distributions. This has a useful converse for ICA: a linear combination $y = \mathbf{w}^T\mathbf{x} = \mathbf{w}^T\mathbf{A}\mathbf{s} = \mathbf{z}^T\mathbf{s}$ (where $\mathbf{z} = \mathbf{A}^T\mathbf{w}$) is a weighted sum of independent non-Gaussian sources. Such a sum is *more Gaussian* than the individual sources unless $\mathbf{z}$ has only one nonzero component — in which case $y = z_k s_k$ is a single source scaled by a constant, which has the same (non-Gaussian) distribution as s_k.

The consequence. To recover one independent component, find the direction $\mathbf{w}$ that makes $\mathbf{w}^T\mathbf{x}$ *as non-Gaussian as possible*. The resulting $\mathbf{w}$ selects exactly one source.

To find all n components, maximize non-Gaussianity repeatedly, constraining each new direction to be orthogonal (in the whitened space) to all previously found ones. This ensures the recovered components are uncorrelated — and, under the ICA model, genuinely independent.

7.2 Kurtosis

The kurtosis of a zero-mean, unit-variance random variable y is:

$$\mathrm{kurt}(y) = \mathbb{E}[y^4] - 3$$

Why -3? For a standard Gaussian, $\mathbb{E}[y^4] = 3$ — a consequence of the moment-generating function. Subtracting 3 normalizes kurtosis to zero for Gaussians and nonzero for most non-Gaussian distributions. For general (not necessarily unit-variance) zero-mean y, the definition is $\mathbb{E}[y^4] - 3(\mathbb{E}[y^2])^2$, which reduces to the formula above when $\mathbb{E}[y^2] = 1$.

Supergaussian vs. subgaussian. Distributions with positive kurtosis are **supergaussian** (leptokurtic) — more peaked at zero with heavier tails than a Gaussian. The Laplace distribution is the canonical example. Distributions with negative kurtosis are **subgaussian** (platykurtic) — flatter near zero with lighter tails. The uniform distribution is the canonical example. Both are exploitable for ICA.

Useful linearity properties (following Hyvärinen and Oja, 2000): for independent x_1 and x_2, and scalar α:

$$\mathrm{kurt}(x_1 + x_2) = \mathrm{kurt}(x_1) + \mathrm{kurt}(x_2), \qquad \mathrm{kurt}(\alpha x_1) = \alpha^4\,\mathrm{kurt}(x_1)$$

These make the optimization landscape analyzable. For $y = z_1 s_1 + z_2 s_2$ with constraint $z_1^2 + z_2^2 = 1$, the kurtosis magnitude is $|z_1^4\,\mathrm{kurt}(s_1) + z_2^4\,\mathrm{kurt}(s_2)|$. This is maximized when exactly one of z_1, z_2 is nonzero — precisely when y equals one source.

Drawbacks. Kurtosis is sensitive to outliers: a single extreme value can dominate the fourth moment. In practice, more robust measures are preferred.

7.3 Negentropy

Setup: Entropy and the Gaussian maximum. The differential entropy of a continuous random vector $\mathbf{y}$ with density $f(\mathbf{y})$ is:

$$H(\mathbf{y}) = -\int f(\mathbf{y}) \log f(\mathbf{y})\, d\mathbf{y}$$

A fundamental result of information theory (Cover and Thomas, 1991) states: *among all continuous random variables with the same covariance matrix, the Gaussian has the largest differential entropy*. The Gaussian is the "most spread out" or "most random" distribution given fixed second-order statistics.

Negentropy. To get a non-Gaussianity measure that is zero for Gaussians and positive otherwise, define:

$$J(\mathbf{y}) = H(\mathbf{y}_{\text{gauss}}) - H(\mathbf{y})$$

where $\mathbf{y}_{\text{gauss}}$ is a Gaussian with the same covariance as $\mathbf{y}$. Since the Gaussian maximizes entropy, $J(\mathbf{y}) \geq 0$ always, with equality if and only if $\mathbf{y}$ is Gaussian. This is the *entropy gap* between the best-case (Gaussian) and the actual distribution.

Negentropy is theoretically optimal — it is the information-theoretically natural measure of non-Gaussianity — and invariant to invertible linear transformations. The problem is computational: computing it requires estimating $f(\mathbf{y})$, which is expensive.

Practical approximations (Hyvärinen, 1998). Based on the maximum-entropy principle, negentropy can be approximated by:

$$J(y) \approx \sum_i k_i \left[\mathbb{E}\{G_i(y)\} - \mathbb{E}\{G_i(\nu)\}\right]^2$$

where $\nu \sim \mathcal{N}(0,1)$, the k_i are positive constants, and G_i are nonquadratic functions chosen for robustness. Two particularly effective choices are:

$$G_1(u) = \frac{1}{a_1}\log\cosh(a_1 u) \quad (a_1 \approx 1), \qquad G_2(u) = -\exp\left(-\frac{u^2}{2}\right)$$

These approximations combine the computational tractability of moment-based measures with the robustness of information-theoretic ones, making them the practical default in ICA.

7.4 Mutual Information and the Infomax Principle

Mutual information between n scalar variables $y_1, \ldots, y_n$ is:

$$I(y_1, \ldots, y_n) = \sum_{i=1}^{n} H(y_i) - H(\mathbf{y})$$

This equals the Kullback-Leibler divergence between the joint density $f(\mathbf{y})$ and the product of marginals $\prod_i f_i(y_i)$ — a natural measure of how much the joint distribution differs from independence. It is always ≥ 0, with equality if and only if the y_i are statistically independent.

ICA as mutual information minimization. Define the ICA transformation as $\mathbf{y} = \mathbf{W}\mathbf{x}$. Minimizing $I(y_1, \ldots, y_n)$ over $\mathbf{W}$ finds the transformation that makes the outputs as independent as possible.

Connection to negentropy. When the y_i are constrained to be uncorrelated and unit-variance (which whitening ensures), it can be shown that:

$$I(y_1, \ldots, y_n) = C - \sum_i J(y_i)$$

where C is a constant independent of $\mathbf{W}$. Therefore *minimizing mutual information is equivalent to maximizing the sum of negentropies*. This gives rigorous justification for the heuristic idea of finding "maximally non-Gaussian" directions.

The Infomax Principle (Bell and Sejnowski, 1995) derives ICA from a neural network perspective: maximize the output entropy of a neural network with nonlinear outputs. Several authors proved that, with nonlinearities chosen as the CDFs of the source densities, this is equivalent to maximum likelihood estimation of the ICA model. So maximum likelihood, mutual information minimization, and infomax are all equivalent formulations of the same optimization — three ways of expressing the same target.

7.5 Preprocessing: Centering and Whitening

Before applying ICA, two preprocessing steps are standard.

Centering. Subtract the sample mean: $\mathbf{x} \leftarrow \mathbf{x} - \mathbb{E}[\mathbf{x}]$. The ICA model can be estimated on centered data; the source means are recovered afterward via $\mathbf{A}^{-1}\mathbb{E}[\mathbf{x}]$.

Whitening. Transform $\mathbf{x}$ to $\tilde{\mathbf{x}}$ such that $\mathbb{E}[\tilde{\mathbf{x}}\tilde{\mathbf{x}}^T] = \mathbf{I}$. Using the PCA transform from Section 4.2, $\tilde{\mathbf{x}} = \mathbf{L}^{-1/2}\mathbf{U}^T\mathbf{x}$ achieves this.

Why whitening helps. After whitening, the transformed mixing matrix $\tilde{\mathbf{A}}$ must satisfy $\tilde{\mathbf{A}}\tilde{\mathbf{A}}^T = \mathbf{I}$ — it must be *orthogonal*. This is because:

$$\mathbb{E}[\tilde{\mathbf{x}}\tilde{\mathbf{x}}^T] = \tilde{\mathbf{A}}\,\mathbb{E}[\mathbf{s}\mathbf{s}^T]\,\tilde{\mathbf{A}}^T = \tilde{\mathbf{A}}\mathbf{I}\tilde{\mathbf{A}}^T = \mathbf{I} \implies \tilde{\mathbf{A}}\tilde{\mathbf{A}}^T = \mathbf{I}$$

where we used the unit-variance assumption on sources. An orthogonal $n \times n$ matrix has $n(n-1)/2$ free parameters (the angles parameterizing n-dimensional rotations), compared to n^2 for a general matrix. **Whitening solves half the ICA problem**: it reduces the search from all invertible matrices to the much smaller space of orthogonal ones. The remaining task — finding the right rotation within this space — is what the ICA algorithm must do.

7.6 The FastICA Algorithm

The FastICA algorithm (Hyvärinen and Oja, 1997) is a fixed-point iteration for finding the unit-weight vector $\mathbf{w}$ that maximizes the negentropy approximation $J(\mathbf{w}^T\tilde{\mathbf{x}})$.

The fixed-point update. At a maximum of $J(\mathbf{w}^T\tilde{\mathbf{x}})$ subject to $\|\mathbf{w}\| = 1$, the first-order optimality conditions (Kuhn-Tucker) together with a Newton-step approximation (exploiting the whitened covariance $\mathbb{E}[\tilde{\mathbf{x}}\tilde{\mathbf{x}}^T] = \mathbf{I}$ to simplify the Jacobian) yield:

$$\mathbf{w}^+ = \mathbb{E}\left\{\tilde{\mathbf{x}}\, g(\mathbf{w}^T\tilde{\mathbf{x}})\right\} - \mathbb{E}\left\{g'(\mathbf{w}^T\tilde{\mathbf{x}})\right\}\mathbf{w}$$

followed by normalization $\mathbf{w} \leftarrow \mathbf{w}^+/\|\mathbf{w}^+\|$, where $g = G'$ is the derivative of the chosen nonquadratic function G. For the two practical choices from Section 7.3:

$$g_1(u) = \tanh(a_1 u), \qquad g_2(u) = u\exp(-u^2/2)$$

The algorithm for one component: initialize $\mathbf{w}$ randomly (unit length), then iterate the update above until $|\mathbf{w}_{\text{new}}^T\mathbf{w}_{\text{old}}| \approx 1$ (new and old $\mathbf{w}$ are colinear modulo sign).

Finding multiple components. To find p components, run the single-component algorithm p times, deflating after each one by projecting out the already-found directions (Gram-Schmidt). Alternatively, use *symmetric decorrelation*: update all p weight vectors simultaneously, then orthogonalize the full matrix via $\mathbf{W} \leftarrow (\mathbf{W}\mathbf{W}^T)^{-1/2}\mathbf{W}$, so no component is privileged over others.

Properties. FastICA converges cubically (vs. linearly for gradient methods), has no step-size parameter to tune, works for both sub- and supergaussian sources with appropriate choice of g, and can estimate components one-by-one when only a subset is needed. It is the standard practical algorithm for ICA in signal processing (EEG/MEG artifact removal), image processing (natural image feature extraction), and financial data analysis.

So what? FastICA operationalizes the entire ICA theory: given centered, whitened data, it iteratively rotates coordinate axes to maximize non-Gaussianity, converging to the demixing matrix that recovers the original independent sources.

8. Sources and Further Reading

Assigned readings:

- **Bishop, *Pattern Recognition and Machine Learning* (2006), Chapter 12.** Primary source for Sections 1–5. Sections 12.1 (PCA), 12.2 (Probabilistic PCA, EM, Bayesian PCA, Factor Analysis), and 12.4.1 (ICA as nonlinear latent variable model). All numbered equations in Sections 2–5 follow Bishop's derivations directly.
- **Hyvärinen and Oja, "Independent Component Analysis: Algorithms and Applications," *Neural Networks* 13(4-5):411–430, 2000.** Primary source for Sections 6–7. The FastICA derivation, negentropy approximations, measures of non-Gaussianity, and the three-formulations equivalence follow this paper closely.
- **Isbell and Viola, "Restructuring Sparse High Dimensional Data for Effective Retrieval," *NIPS* 1999.** Assigned reading; provides empirical illustration of the PCA vs. ICA distinction through text retrieval. The key observation — that ICA axes yield highly kurtotic (non-Gaussian) document projections while PCA axes yield near-Gaussian ones — illustrates Section 7.1 in a concrete application domain.

Key results and their attributions:

- The PCA derivation as a constrained eigenvector problem (Sections 2–3) is standard; the treatment here follows Bishop §12.1.
- The maximum likelihood solution for PPCA (Section 5.4) — including the closed-form $\mathbf{W}_{\text{ML}}$ and the result that σ^2_{ML} equals the average of discarded eigenvalues — was proved by Tipping and Bishop (1999b). The result that all other stationary points are saddle points is also from Tipping and Bishop (1999b). A related conjecture was made independently by Roweis (1998).
- The result that Gaussian latents are unidentifiable in ICA (Section 6.5) is standard; the proof via rotational symmetry follows Hyvärinen and Oja §3.3.
- The infomax principle was introduced by Bell and Sejnowski (1995); equivalence between infomax and maximum likelihood was proved by Cardoso (1997) and Pearlmutter and Parra (1997).
- The FastICA fixed-point algorithm and its convergence analysis are from Hyvärinen (1999a); the connection to maximum likelihood is proved in Hyvärinen (1999b).
- The additivity of kurtosis for independent variables and the kurtosis-based optimization landscape analysis (Section 7.2) follow Delfosse and Loubaton (1995), cited in Hyvärinen and Oja.
- Results in Section 5.5 for the PPCA posterior follow the general linear-Gaussian result of Bishop §2.3 (equation 2.116); the derivation is not reproduced here.

For deeper reading:

- **Tipping and Bishop (1999b), "Probabilistic Principal Component Analysis," *J. Royal Statistical Society B*.** The original PPCA paper; contains the proof of the closed-form ML solution and the saddle-point characterization.
- **Bell and Sejnowski (1995), "An Information-Maximization Approach to Blind Separation and Blind Deconvolution," *Neural Computation* 7:1129–1159.** The original infomax ICA paper.
- **Comon (1994), "Independent Component Analysis — A New Concept?", *Signal Processing* 36:287–314.** Provides the formal definition of ICA and proves identifiability for non-Gaussian sources.
- **Cover and Thomas, *Elements of Information Theory*, 2nd ed. (2006).** Standard reference for the maximum-entropy property of the Gaussian (Section 7.3) and the properties of differential entropy and mutual information used throughout Section 7.